

République du Cameroun

Republic du Cameroon

Paix – Travail – Patrie

Peace – Work – Fatherland

Ministère de l'Education Secondaire

Ministry of Secondary Education

Institut Universitaire Saint Jean

Saint Jean Institute



Thème :

CAHIER D'ANALYSE & CONCEPTION : MYPING

Supervisé par:

Mr. NGUIMBUS Emmanuel

Fait par :

- ESSAMA AXEL MERLIN
- MENGA MARCO ROMUALD
- MOUGANG NGUETI YAN KEMUEL
- TAGNE JOSEPH NATHANAEL
- TCHATCHOUA NGASSAM GIOVANNI NESTOR

GLOSSAIRE

Terme	Définition
ICMP	Internet Control Message Protocol. Protocole de couche réseau utilisé pour transporter des messages de contrôle et d'erreur entre machines IP. Défini par la RFC 792 (1981)
RTT	Round-Trip Time. Durée totale d'un aller-retour d'un paquet entre la source et la destination, exprimée en millisecondes
TTL	Time To Live. Compteur dans l'en-tête IP qui est décrémenté à chaque routeur traversé. Quand il arrive à 0, le paquet est détruit et un message ICMP Time Exceeded est renvoyé
Raw Socket	Socket qui permet l'accès direct à la couche IP, sans passer par TCP ou UDP. Permet de construire des paquets entièrement à la main, mais demande des privilèges élevés
Checksum	Somme de contrôle. Valeur calculée sur les données d'un paquet, qui permet de détecter les erreurs de transmission. Le récepteur recalcule la valeur et la compare à celle reçue
mdev	Mean Deviation. Écart-type des RTT mesurés sur la session. Indicateur de la stabilité du réseau : plus il est faible, plus les RTT sont réguliers
CLI	Command Line Interface. Interface en ligne de commande, par opposition à une interface graphique

Terme	Définition
DNS	Domain Name System. Système qui traduit les noms de domaine (google.com) en adresses IP (142.250.74.78)
SIGINT	Signal d'interruption sous Unix. Envoyé par la combinaison Ctrl+C, il demande au processus de s'arrêter proprement
CAP_NET_RAW	Capability Linux qui autorise un processus à ouvrir des raw sockets sans avoir les pleins privilèges root
RFC 792	Document de référence qui définit le protocole ICMP, publié par l'IETF en septembre 1981

LISTE DE TABLEAUX

Tableau 1 : Champ d'en-tête ICMP	8
Tableau 2 : Options connue par le programme	10
Tableau 3 : Critère de sélection du langage	14
Tableau 4 : Module présent dans le programme	15
Tableau 5 : Plan de test	19
Tableau 6 : Repartition de l'équipe.....	21
Tableau 7 : Répartition des fichiers du programme	22
Tableau 8 : Planning Prévisionnelle.....	23
Tableau 9 : Analyse des risques	26
Tableau 10 : Ensemble des ecarts	28

SOMMAIRE

GLOSSAIRE	i
LISTE DE TABLEAUX.....	iii
1. Introduction et présentation du projet	5
2. Analyse de l'existant	7
3. Spécifications fonctionnelles.....	9
4. Spécifications non fonctionnelles	12
5. Architecture et conception.....	14
6. Plan de tests et validation	19
7. Organisation et planification	21
8. Livrables attendus.....	25
9. Identification et gestion des risques	26
10. Écarts entre le cahier des charges initial et la réalisation	27
12. Références bibliographiques	30

1. Introduction et présentation du projet

1.1 Contexte

Le projet myping a été lancé dans le cadre du cours de « Théorie des Systèmes d'Exploitation ». L'idée de départ vient d'un constat assez simple : on parle souvent du protocole ICMP en cours, on l'illustre avec quelques captures Wireshark, mais on n'a jamais vraiment vu ce qui se passe sous le capot quand on tape une commande ping dans un terminal. Notre enseignant a donc proposé qu'on retroussé les manches et qu'on refasse ping nous-mêmes, en partant de zéro.

La commande ping, c'est l'un des outils les plus basiques et les plus utilisés en réseau. Tout le monde s'en sert, des administrateurs système aux développeurs en passant par les étudiants en TP. Son principe paraît trivial : on envoie un paquet, on attend la réponse, on mesure le temps. Mais quand on commence à regarder de plus près, on se rend compte qu'il y a pas mal de subtilités, surtout au niveau des raw sockets et de la construction des paquets ICMP.

Notre version, qu'on a appelée myping, ne cherche pas à remplacer ping. L'objectif est plutôt pédagogique : reconstruire l'outil pour comprendre comment il fonctionne, et pouvoir expliquer ligne par ligne ce que fait notre code.

1.2 Objectifs du projet

On s'est fixé plusieurs objectifs au démarrage du projet :

- Comprendre concrètement le protocole ICMP, et notamment le format des paquets Echo Request et Echo Reply
- Apprendre à manipuler les raw sockets sous Linux, ce qui nécessite des privilèges particuliers
- Produire un outil en ligne de commande qui se comporte comme ping natif sur les options principales
- Calculer et afficher des statistiques réseau pertinentes : RTT min/max/moyenne/écart-type, taux de perte
- Gérer proprement les cas limites et les erreurs (hôte injoignable, Ctrl+C, timeout, etc.)

L'idée derrière tout ça, ce n'est pas juste de "faire fonctionner" myping. C'est de comprendre pourquoi chaque ligne du code est là, et de pouvoir maintenir le projet sur le long terme.

1.3 Périmètre

Le périmètre a été fixé après une réunion de cadrage avec l'équipe. Voici ce qu'on s'est engagés à livrer :

- Implémentation du protocole ICMP Echo Request et Echo Reply (RFC 792)
- Support des options classiques : -c, -i, -s, -t, -w, -q, -n, -h
- Calcul et affichage des statistiques réseau (RTT min, moyenne, max, écart-type, pertes)
- Gestion robuste des erreurs réseau et des cas limites
- Interface en ligne de commande dans le style Unix classique

On a volontairement laissé de côté certains aspects : pas de support IPv6 (ICMPv6), pas de portage macOS ou Windows, pas d'export de données dans un fichier. Ce sont des extensions qu'on pourra ajouter plus tard si on reprend le projet.

2. Analyse de l'existant

2.1 Un outil ancien, toujours utilisé

La commande ping a été écrite par Mike Muuss en 1983, alors qu'il travaillait au Ballistic Research Laboratory de l'armée américaine. L'histoire est sympa : il s'est inspiré du principe du sonar sous-marin, qui envoie un signal acoustique et attend l'écho pour détecter un obstacle. Le nom "Packet Internet Groper" lui a été attribué après coup, par rétro-acronymie, pour faire correspondre PING à un sigle technique.

Plus de quarante ans plus tard, ping est toujours là, sur tous les systèmes d'exploitation : Linux, macOS, Windows, BSD, Android, iOS. Le comportement de base reste le même partout, mais les implémentations diffèrent un peu selon les plateformes (notamment dans le format d'affichage et les options disponibles). C'est en partie pour cette raison qu'on a choisi de cibler exclusivement Linux pour notre projet : on s'évite les problèmes de portabilité et on peut se concentrer sur la logique métier.

2.2 Le protocole ICMP en bref

Le protocole ICMP (Internet Control Message Protocol) est défini par la RFC 792, publiée par Jon Postel en septembre 1981. Il opère à la couche réseau (couche 3 du modèle OSI) et il est encapsulé directement dans les datagrammes IP, sans passer par TCP ou UDP.

ICMP est conçu pour transporter des messages de contrôle et d'erreur entre machines : "destination unreachable", "time exceeded", redirection, etc. Pour notre projet, on s'intéresse à deux types de messages bien précis :

- Type 8 (Echo Request) : c'est le paquet qu'on envoie à la machine distante
- Type 0 (Echo Reply) : c'est la réponse qu'elle nous renvoie

Le format d'un paquet ICMP Echo est le suivant :

Champ	Taille	Description
Type	8 bits	8 pour Echo Request, 0 pour Echo Reply
Code	8 bits	Sous-type, toujours 0 pour les messages Echo
Checksum	16 bits	Somme de contrôle calculée sur tout l'en-tête ICMP

Champ	Taille	Description
Identifier	16 bits	Identifiant de session, généralement le PID du processus
Sequence	16 bits	Numéro de séquence du paquet (1, 2, 3...)
Data	Variable	Charge utile, contient souvent un timestamp d'envoi

Tableau 1 : Champ d'en-tête ICMP

Un point qu'on a découvert un peu douloureusement pendant le développement : le calcul du checksum ICMP. C'est un complément à un sur 16 bits, calculé sur l'ensemble de l'en-tête ICMP et de la charge utile. Si on se trompe dans ce calcul, le paquet est silencieusement rejeté par la machine distante. Et silencieusement, ça veut vraiment dire silencieusement : pas de message d'erreur, pas de log, rien. On peut chercher pendant des heures pourquoi rien ne répond, alors que c'est juste un bit mal placé dans la somme de contrôle.

2.3 Pourquoi refaire ping ?

La question est légitime. La commande ping existe depuis quarante ans, elle marche, elle est dispo partout. Pourquoi se fatiguer à en refaire une version ?

Plusieurs raisons nous ont poussés à le faire :

- Le code source de `iputils` (la version Linux moderne de ping) est assez touffu pour un débutant. Il gère IPv4, IPv6, plein de cas particuliers, et c'est difficile à lire quand on découvre les raw sockets
- Refaire un outil qu'on utilise tous les jours est une démarche pédagogique très efficace. On voit concrètement le décalage entre "je sais ce que ça fait" et "je sais comment ça fait"
- C'est l'occasion idéale de toucher à des choses qu'on n'aborde pas souvent en cours : sérialisation manuelle de paquets, calcul de checksum, gestion de signaux Unix, raw sockets, etc.

Au final, on récupère un projet entièrement maîtrisé, qu'on peut faire évoluer comme on veut, et qui nous a fait progresser sur des sujets concrets.

3. Spécifications fonctionnelles

3.1 Construction et envoi des paquets ICMP

C'est le cœur du projet. myping doit être capable de construire un paquet ICMP Echo Request entièrement à la main, en-tête comprise, et de l'envoyer via un raw socket à une adresse IP ou à un nom d'hôte donné en paramètre.

Concrètement, le programme doit :

- Accepter en argument soit une adresse IP (par exemple 8.8.8.8), soit un nom de domaine (par exemple google.com)
- Résoudre le nom de domaine en adresse IP via une requête DNS (fonction getaddrinfo)
- Afficher à l'utilisateur l'IP résolue, pour qu'il sache à quoi correspond son ping
- Construire l'en-tête ICMP (type, code, identifier, sequence) et insérer un timestamp dans le payload
- Calculer le checksum sur l'ensemble du paquet
- Envoyer le paquet via un raw socket configuré avec IPPROTO_ICMP

Comme l'ouverture d'un raw socket demande des privilèges particuliers (CAP_NET_RAW sous Linux), il faut lancer le programme avec sudo. C'est une contrainte forte qu'on a choisi d'accepter, plutôt que de mettre en place un mécanisme de gestion fine des capacités.

3.2 Réception et traitement des réponses

Une fois le paquet envoyé, on attend la réponse. Le timeout par défaut est d'une seconde (configurable via -w). À la réception d'un paquet, le programme doit :

- Vérifier que c'est bien un Echo Reply (type 0). Sinon, on l'ignore
- Vérifier que l'identifier correspond bien au PID du processus, pour ne pas confondre avec un paquet destiné à un autre ping qui tournerait en parallèle
- Extraire le timestamp inséré au moment de l'envoi, contenu dans le payload du paquet
- Calculer le RTT (Round-Trip Time) en faisant la différence entre l'heure courante et le timestamp d'envoi
- Mettre à jour les compteurs et statistiques globaux (min, max, moyenne, écart-type)
- Afficher une ligne de résultat dans le format attendu

Si aucun paquet n'arrive avant l'expiration du timeout, on affiche "Pas de réponse pour icmp_seq=N" et on passe au paquet suivant.

3.3 Options de la ligne de commande

L'outil supporte les options suivantes, qui correspondent au comportement de ping natif sous Linux dans les grandes lignes :

Option	Paramètre	Description
-c	nombre	Nombre de paquets à envoyer (illimité par défaut)
-i	secondes	Intervalle entre deux envois (défaut : 1.0 s)
-s	octets	Taille de la charge utile (défaut : 56 octets)
-t	valeur	TTL (Time To Live) des paquets IP
-w	secondes	Durée maximale totale du ping
-q	(aucun)	Mode silencieux : affiche uniquement le résumé final
-n	(aucun)	Désactive la résolution DNS inverse
-h	(aucun)	Affiche l'aide et quitte

Tableau 2 : Options connues par le programme

On a fait un choix qui diffère légèrement de ping natif : on utilise -w en minuscule pour la durée totale (alors que ping natif utilise -W pour le timeout par paquet et -w pour la durée totale). C'est une simplification qu'on assume.

3.4 Affichage des statistiques

À la fin de l'exécution, que ce soit parce que le quota -c est atteint, parce que la durée -w est dépassée, ou parce que l'utilisateur a fait Ctrl+C, on affiche un récapitulatif de session. Le format est volontairement proche de celui de ping natif pour faciliter les comparaisons. Une différence : on a tout traduit en français parce que toute l'équipe est francophone et qu'on trouve ça plus naturel.

Pour chaque paquet reçu, l'affichage ressemble à :

```
64 octets de google.com (142.250.74.78): icmp_seq=0 ttl=117 temps=12.348 ms
```

À la fin de la session, le bloc statistiques affiche :

```
--- google.com ping statistiques ---  
5 paquets transmis, 5 recus, 0% perte, duree 4012 ms  
rtt min/moy/max/mdev = 10.234/12.107/15.812/1.927 ms
```

3.5 Extensions envisagées (non livrées)

On avait identifié plusieurs extensions intéressantes au démarrage du projet. Le calendrier ne nous a pas permis de toutes les traiter. On les liste ici pour mémoire, ce sont des pistes d'évolution si on reprend le projet plus tard :

- Support de IPv6 via le protocole ICMPv6 (paquets Echo Request type 128, Echo Reply type 129)
- Mode flood (-f) pour envoyer des paquets en rafale, utilisé pour les tests de charge
- Mode verbeux (-v) pour afficher plus d'informations de debug
- Export des résultats au format CSV ou JSON, pour faciliter l'analyse en post-traitement
- Résolution DNS inverse en arrière-plan, pour ne pas bloquer l'affichage de la première réponse

4. Spécifications non fonctionnelles

4.1 Performance

La précision de mesure du RTT est le critère de performance le plus important pour un outil comme ping. On vise une erreur de mesure inférieure à la milliseconde dans des conditions normales d'utilisation.

Pour ça, on utilise la fonction `gettimeofday()` qui offre une précision en microsecondes. C'est largement suffisant pour les RTT typiques en environnement réseau (de quelques ms à quelques centaines de ms). On avait initialement envisagé d'utiliser `clock_gettime()` avec l'horloge `CLOCK_MONOTONIC`, qui offre une précision nanoseconde et n'est pas affectée par les ajustements de l'horloge système. On a finalement gardé `gettimeofday` parce que ça simplifie le code et que la précision est largement suffisante pour notre usage. Cet écart est documenté dans la section dédiée.

4.2 Fiabilité et robustesse

Un programme qui parle au réseau doit savoir gérer l'inattendu. C'est un point auquel on a fait particulièrement attention :

- Gestion propre du signal SIGINT (Ctrl+C) : on attrape le signal, on met fin à la boucle principale, et on affiche les statistiques avant de quitter
- Pas de blocage indéfini : la réception utilise `select()` avec un timeout, on n'attend jamais sans fin
- Validation des arguments en ligne de commande via `getopt()`, avec affichage de l'aide en cas d'erreur
- Gestion explicite des erreurs réseau : hôte inaccessible, échec de la résolution DNS, erreur d'envoi
- Vérification systématique de l'identifier et du type des paquets reçus, pour éviter les confusions

4.3 Portabilité

On cible Linux uniquement, avec un noyau 4.x ou plus récent. Le code utilise des en-têtes spécifiques à Linux (notamment `netinet/ip_icmp.h`, `netinet/ip.h`), donc il ne compilera pas tel quel sur macOS ou les systèmes BSD. Aucune bibliothèque tierce n'est nécessaire, à part la bibliothèque mathématique standard (`libm`, pour la fonction `sqrt` utilisée dans le calcul de l'écart-type). Cette dépendance est explicitement gérée dans le `Makefile`.

4.4 Sécurité

L'utilisation des raw sockets demande des privilèges élevés sous Linux. On a réfléchi à mettre en place un mécanisme de drop de privilèges (le programme tourne en root juste le temps d'ouvrir le socket, puis abandonne ses droits via `setuid/setgid`). Au final, on a gardé une approche plus simple : `sudo` tout le long de l'exécution. Pour un déploiement sérieux, ce point mériterait d'être revu.

Côté code, on a évité les fonctions dangereuses (pas de `gets`, pas de `strcpy` non bornée), on utilise `strncpy` avec une taille explicite, et on libère systématiquement la mémoire allouée pour les paquets.

4.5 Maintenabilité

Pour que le code reste lisible et que d'autres puissent le reprendre, on a appliqué quelques principes :

- Découpage en modules cohérents : un fichier par responsabilité, et un seul header partagé (`commun.h`)
- Commentaires en français, parce que tout le monde dans l'équipe l'est, et que ça nous a paru plus naturel à relire
- Conventions de nommage uniformes : noms de variables et de fonctions en français, en minuscules avec underscore
- `Makefile` minimal mais fonctionnel, qui gère la compilation et le nettoyage
- Un fichier `README.md` documentera la compilation et l'utilisation sur le dépôt GitHub

5. Architecture et conception

5.1 Choix du langage

Le choix du langage a été la première décision importante du projet. On a hésité entre deux options : le C et Python. Voici comment on les a comparés :

Critère	C	Python
Performance	Excellente, accès bas niveau direct	Correcte via socket.SOCK_RAW
Complexité	Plus élevée, gestion mémoire manuelle	Plus simple, syntaxe haut niveau
Valeur pédagogique	Très forte pour la prog système	Modérée, masque trop de détails
Disponibilité d'outils	GCC, Make, GDB, Valgrind	Interpréteur standard, scapy
Portabilité Linux	Excellente	Très bonne

Tableau 3 : Critère de sélection du langage

On a choisi le C, et ce choix s'est imposé assez vite. La raison principale, c'est l'intérêt pédagogique : construire à la main les structures ICMP, calculer le checksum nous-mêmes, gérer la mémoire avec malloc et free, manipuler les sockets bruts. Tout ça constitue une expérience d'apprentissage qu'on ne retrouverait pas avec Python, où la majorité du travail bas niveau est cachée derrière des abstractions.

Et accessoirement, on est dans un cours de programmation système. Faire ce projet en Python aurait été un peu paradoxal.

5.2 Décomposition en modules

Le projet est structuré en sept fichiers source plus un en-tête partagé. Le principe directeur : un fichier correspond à une responsabilité claire, et toutes les fonctions partagées sont déclarées dans commun.h.

Fichier	Responsabilité
main.c	Point d'entrée du programme. Parsing des arguments avec getopt, initialisation, appel des autres modules, boucle principale d'envoi/réception
commun.h	En-tête partagé. Bibliothèques système, constantes (taille par défaut, timeout), prototypes des fonctions, déclarations extern des variables globales
controle_envoi.c	Logique de fin de boucle : vérification de la deadline (option -w), vérification du quota de paquets (option -c), gestion de la pause entre envois (option -i)
ICMP.c	Construction du paquet ICMP Echo Request : allocation du buffer, remplissage de l'en-tête, insertion du timestamp, calcul et insertion du checksum
reseau.c	Couche réseau : ouverture du raw socket avec configuration du TTL, envoi des paquets, réception avec décodage des en-têtes IP et ICMP, résolution DNS inverse
format_ip.c	Résolution DNS du nom d'hôte vers son adresse IP via getaddrinfo, conversion en chaîne lisible avec inet_ntop
affichage.c	Affichage utilisateur : message d'aide (-h), bannière de démarrage, statistiques finales (paquets, perte, RTT min/moyenne/max/écart-type)
calcul.c	Définition des variables globales partagées, handler du signal SIGINT, calcul du temps écoulé en millisecondes, calcul du checksum ICMP

Tableau 4 : Module présent dans le programme

On avait initialement prévu un module séparé pour les statistiques (stats.c). En pratique, les variables d'accumulation sont déclarées dans calcul.c avec les autres globales, et leur affichage est dans affichage.c. C'est une petite entorse au principe "un fichier, une responsabilité", mais l'ensemble reste lisible.

5.3 Variables globales partagées

On a choisi d'utiliser quelques variables globales pour les compteurs et les accumulateurs de RTT. C'est généralement déconseillé en bonne pratique logicielle, mais dans notre cas ça simplifie pas mal le code : on n'a pas à trimballer une structure de statistiques en paramètre de toutes les fonctions, et comme le programme est mono-thread, il n'y a pas de problème de concurrence.

Les variables globales sont :

- `g_continuer` : booléen qui passe à 0 sur réception de SIGINT, ce qui termine la boucle principale
- `g_nb_envoyes`, `g_nb_recus` : compteurs de paquets
- `g_rtt_min`, `g_rtt_max` : RTT extrêmes mesurés sur la session
- `g_somme_rtt`, `g_somme_rtt_carre` : accumulateurs pour calculer la moyenne et l'écart-type
- `g_nom_destination` : nom de la cible, utilisé dans l'affichage final

Elles sont toutes définies dans `calcul.c` et déclarées `extern` dans `commun.h` pour être utilisables depuis les autres modules.

5.4 Flux d'exécution

Voici le déroulement chronologique du programme, de son lancement à sa terminaison :

1. Parsing des arguments en ligne de commande avec `getopt`, application des valeurs par défaut pour les options non précisées
2. Récupération de la destination (dernier argument non-option), vérification qu'elle est bien présente
3. Résolution DNS de la destination via `getaddrinfo` (fonction `obtenir_ip`)
4. Ouverture du raw socket avec `IPPROTO_ICMP`, application du TTL si demandé (fonction `ouvrir_socket`)
5. Installation du handler de signal pour SIGINT, qui pointera vers `arreter_le_ping`
6. Affichage du message de démarrage (`afficher_debut`)
7. Entrée dans la boucle principale, tant que `g_continuer` vaut 1
8. À chaque itération : vérification de la deadline, vérification du quota, construction du paquet ICMP, envoi, réception avec timeout, mise à jour des statistiques, pause
9. Sortie de la boucle (Ctrl+C, quota atteint, ou deadline expirée)
10. Fermeture du socket

11. Affichage des statistiques finales (afficher_resultats)

12. Retour avec code 0 si au moins un paquet a été reçu, sinon 1

5.5 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présente les interactions entre l'utilisateur (généralement un administrateur réseau ou un développeur) et le système myping. L'acteur principal a accès aux cas d'utilisation suivants :

- Pinger une cible (par adresse IP ou par nom de domaine)
- Configurer le nombre de paquets à envoyer (-c)
- Configurer la taille des paquets et le TTL (-s, -t)
- Configurer l'intervalle d'envoi et la durée maximale (-i, -w)
- Choisir le mode d'affichage (silencieux -q, sans DNS -n)
- Consulter l'aide (-h)
- Interrompre le ping en cours (Ctrl+C) et obtenir le récapitulatif

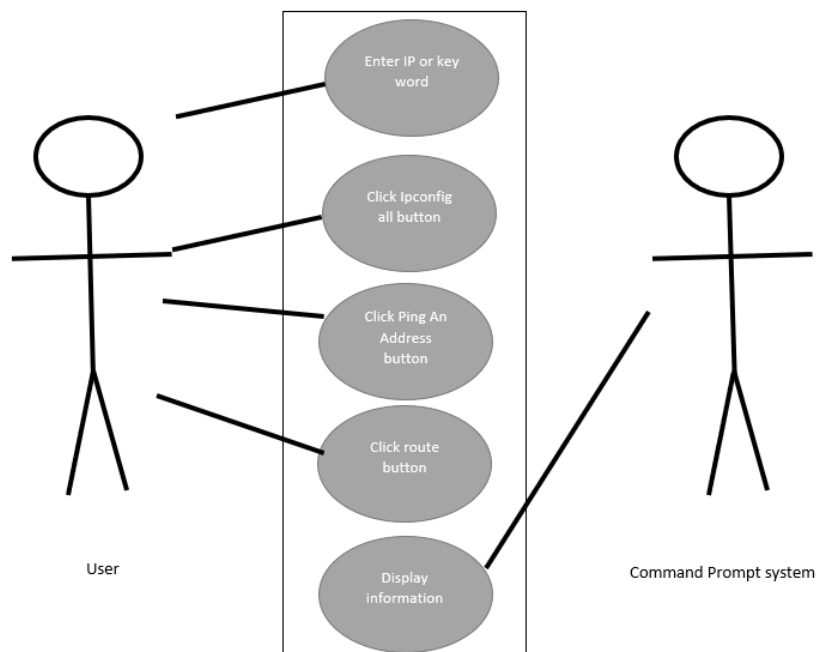


Figure 1 : Diagramme de Cas d'Utilisation

5.6 Diagramme de séquence

Le diagramme de séquence détaille les échanges entre les trois acteurs du système : l'utilisateur qui lance la commande, le processus myping qui s'exécute localement, et la machine cible distante avec laquelle on communique en ICMP. Il met en évidence les phases suivantes : la résolution DNS préalable, l'ouverture du raw socket, la boucle d'envoi/réception, et l'affichage final des statistiques après réception du signal SIGINT ou atteinte des conditions de fin.

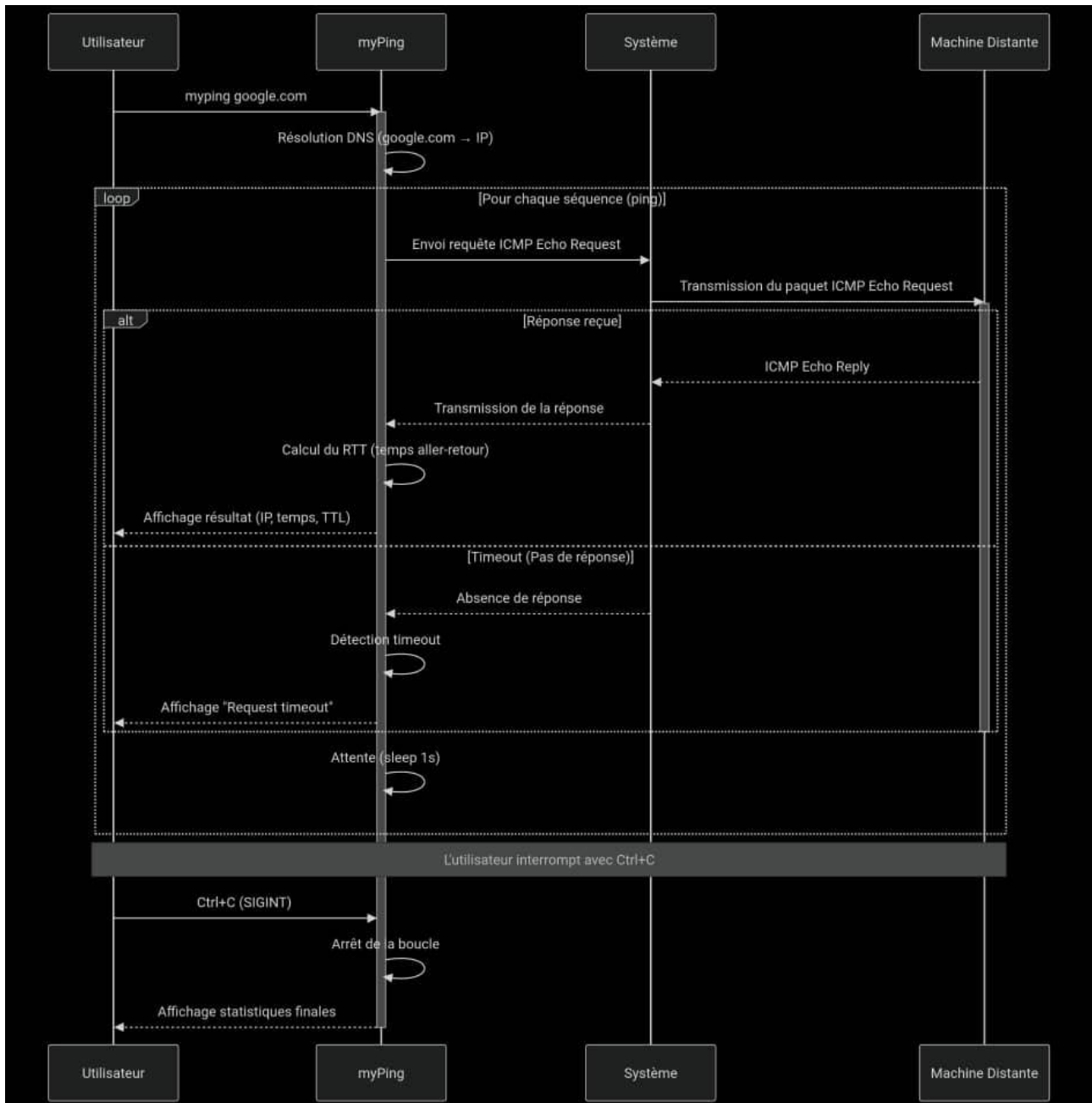


Figure 2 : Diagramme de Séquence

6. Plan de tests et validation

6.1 Tests fonctionnels

Pour valider que chaque fonctionnalité fait bien ce qu'on attend d'elle, on a défini une série de cas de test. Chaque cas vérifie un comportement précis et a un résultat attendu connu à l'avance.

N°	Cas testé	Commande	Résultat attendu
T01	Ping IP publique	<code>sudo ./myping 8.8.8.8</code>	Réponses ICMP avec RTT affiché
T02	Résolution DNS	<code>sudo ./myping google.com</code>	Résolution OK puis ping fonctionnel
T03	Limite avec -c	<code>sudo ./myping -c 5 8.8.8.8</code>	Exactement 5 paquets envoyés
T04	Intervalle court	<code>sudo ./myping -i 0.5 -c 4 8.8.8.8</code>	Envoi toutes les 500 ms
T05	TTL faible	<code>sudo ./myping -t 2 8.8.8.8</code>	TTL exceeded probable
T06	Taille payload	<code>sudo ./myping -s 128 -c 3 8.8.8.8</code>	Paquet de 136 octets total
T07	Hôte inexistant	<code>sudo ./myping hote.bidon.xyz</code>	Erreur DNS, code retour non nul
T08	Interruption	<code>sudo ./myping 8.8.8.8 + Ctrl+C</code>	Statistiques affichées avant sortie
T09	Mode silencieux	<code>sudo ./myping -q -c 5 8.8.8.8</code>	Que les stats finales
T10	Loopback	<code>sudo ./myping 127.0.0.1</code>	Réponses très rapides, RTT proche de 0

Tableau 5 : Plan de test

6.2 Tests de robustesse

On a aussi vérifié le comportement du programme dans des cas limites :

- Lancement sans argument de destination : message d'erreur explicite, affichage de l'aide, code retour 2
- Argument invalide (-c abc, -i toto) : getopt détecte, message d'erreur, affichage de l'aide
- Lancement sans sudo : erreur explicite avec indication qu'il faut sudo
- Durée -w 1 sur un host lointain : pas de blocage, sortie propre au bout d'une seconde
- Combinaisons d'options (par exemple -c 10 -i 0.1 -s 200) : comportement attendu

6.3 Comparaison avec ping natif

Un de nos critères de validation les plus importants, c'est la cohérence avec la commande ping native. On a lancé en parallèle ping et myping sur les mêmes cibles, et on a comparé :

- Les RTT individuels : doivent être dans la même plage, sachant qu'il y a toujours une variabilité réseau
- Le pourcentage de perte : doit être identique ou très proche
- Les statistiques min/moyenne/max : doivent être du même ordre de grandeur
- L'écart-type (mdev) : doit avoir des valeurs similaires

Les résultats obtenus sur des cibles stables (loopback, gateway locale, serveurs DNS publics comme 8.8.8.8 ou 1.1.1.1) montrent que myping produit des mesures cohérentes avec ping natif.

7. Organisation et planification

7.1 Composition de l'équipe

L'équipe 10 est composée de cinq étudiants de 4e année du cycle ingénieur, spécialité Sécurité Réseaux et Télécommunications. Voici la répartition des rôles pour ce projet :

Membre	Rôle dans le projet
Tagne Joseph	Chef de projet, diagramme de cas d'utilisation, développement de main.c, affichage.c, format_ip.c, Makefile et commun.h
Tchatchoua Jiovanni	Présentation (introduction côté commande ping/myping), développement de controle_envoi.c
Mougang Yann	Présentation (introduction côté protocole ICMP), développement de ICMP.c
Essama Meryl	Développement de calcul.c (variables globales, handler signal, calcul du checksum et du RTT)
Menga Marco	Diagramme de séquence, développement de reseau.c (raw socket, envoi, réception)

Tableau 6 : Repartition de l'équipe

7.2 Répartition par fichier

Pour récapituler, voici l'attribution des fichiers de code :

Fichier	Responsable	Lignes (env.)
main.c	Tagne Joseph	~140
commun.h	Tagne Joseph	~75
affichage.c	Tagne Joseph	~80
format_ip.c	Tagne Joseph	~40
Makefile	Tagne Joseph	~60
controle_envoi.c	Tchatchoua Giovanni	~45
ICMP.c	Mougang Yann	~35
calcul.c	Essama Meryl	~55
reseau.c	Menga Marco	~155

Tableau 7 : Répartition des fichiers du programme

7.3 Planning prévisionnel

Le projet s'est étalé sur environ cinq semaines, avec les phases suivantes :

Phase	Activités	Durée
Phase 1	Analyse du sujet, choix techniques, rédaction du cahier des charges initial	1 semaine
Phase 2	Mise en place de l'environnement, structure du projet, premier Makefile	3 jours
Phase 3	Implémentation du cœur : raw socket, construction ICMP, envoi/réception basique	1 semaine
Phase 4	Ajout des options en ligne de commande, module statistiques, affichage formaté	1 semaine
Phase 5	Tests, débogage, comparaison avec ping natif, ajustements	4 jours
Phase 6	Rédaction des livrables (dossier, rapport, présentation), préparation de la soutenance	3 jours

Tableau 8 : Planning Prévisionnelle

7.4 Outils et environnement

On a utilisé un ensemble d'outils standard pour ce type de projet :

- Système d'exploitation : Linux (distribution Ubuntu pour la majorité de l'équipe)
- Compilateur : GCC avec les options `-Wall -O2` pour avoir tous les avertissements et une optimisation correcte
- Outil de build : Make
- Éditeurs : Visual Studio Code principalement, nano et vim pour les retouches rapides
- Inspection réseau : Wireshark pour visualiser les paquets ICMP réellement envoyés sur l'interface
- Débogage : GDB pour les segfaults, printf bien placés pour le reste
- Gestion de version : Git, avec un dépôt partagé sur GitHub
- Communication d'équipe : WhatsApp pour le quotidien, réunions en présentiel pour les points d'étape

8. Livrables attendus

8.1 Code source

- Code source complet compilable via une simple commande make
- Makefile avec les cibles all et clean
- Code commenté en français, structure modulaire claire
- Dépôt Git public hébergé sur GitHub

8.2 Documentation

- README.md décrivant la compilation, l'installation et des exemples d'utilisation
- Commentaires inline dans le code sur les fonctions et structures non triviales
- Manuel d'utilisation accessible en ligne via l'option -h

8.3 Documents

- Ce dossier d'analyse et de conception
- Un rapport technique détaillé, axé sur les choix d'implémentation, les difficultés rencontrées et les résultats obtenus
- Un support de présentation pour la soutenance et la démonstration

9. Identification et gestion des risques

Pour ne pas se laisser surprendre, on a identifié dès le début du projet plusieurs risques principaux, avec pour chacun une stratégie de mitigation :

Risque	Probabilité	Impact	Mesure de mitigation
Difficultés avec les raw sockets (privilèges, configuration)	Élevée	Élevé	Tester très tôt avec un setup minimal sur loopback
Erreur dans le calcul du checksum ICMP	Moyenne	Élevé	Validation visuelle avec Wireshark pour comparer avec ping natif
Mauvaise interprétation des champs réseau (boutisme, alignement)	Moyenne	Moyen	Utilisation systématique de htons / ntohs, tests sur cibles connues
Dépassement du planning	Moyenne	Moyen	Buffer de 3 jours prévu en fin de phase 5
Bug sur les exécutions longues (fuite mémoire, débordement)	Faible	Faible	Tests manuels avec plusieurs milliers de paquets
Conflit de version sur le dépôt Git	Faible	Faible	Convention de branches par développeur, merge régulier

Tableau 9 : Analyse des risques

10. Écarts entre le cahier des charges initial et la réalisation

Cette section est importante : elle recense honnêtement les écarts entre ce qu'on avait prévu au démarrage du projet et ce qu'on a effectivement livré. C'est un exercice de transparence, mais aussi de pédagogie : ces écarts sont souvent plus instructifs que les choses qui se sont passées comme prévu.

10.1 Vue d'ensemble des écarts

Élément prévu initialement	Réalisation effective	Justification
Découpage en 5 modules (args, icmp, network, stats, display)	Découpage en 7 modules en français (controle_envoi, ICMP, reseau, format_ip, affichage, calcul, main)	Notre découpage s'est imposé naturellement pendant l'implémentation. Les statistiques se sont retrouvées partagées entre calcul.c (accumulateurs) et affichage.c (présentation), au lieu d'un module unique
Fonction temporelle : clock_gettime + CLOCK_MONOTONIC (précision ns)	gettimeofday (précision µs)	Précision microseconde suffisante pour les RTT typiques (> 1 ms). clock_gettime aurait été plus rigoureux mais l'écart pratique est négligeable pour notre cas d'usage
Option -v (mode verbeux)	Non implémentée. Remplacée par -n (désactivation du DNS inverse)	On s'est rendu compte que -n était plus utile en pratique (gain de temps évident sur des cibles distantes). Le mode

Élément prévu initialement	Réalisation effective	Justification
		verbeux n'apportait pas grand-chose sur notre périmètre
Cibles Makefile : all, clean, fclean, re, test	Cibles : all, clean	Cibles minimales suffisantes pour notre flux de développement. fclean et re sont des conventions 42 dont on n'avait pas besoin
Drop de privilèges via setuid / setgid	Non implémenté, sudo tout au long	Complexité disproportionnée pour le cadre du projet. À garder en tête pour un éventuel déploiement réel
Tests Valgrind systématiques sur exécutions > 10 000 paquets	Tests manuels par lancements longs, sans Valgrind systématique	Pas eu le temps de mettre en place une vraie campagne Valgrind. Le code libère pourtant la mémoire allouée à chaque paquet, donc le risque de fuite est limité
Format d'affichage en anglais (style ping iputils)	Affichage en français (PING ... octets de donnees, perte, etc.)	Choix assumé : on code en français, on commente en français, on affiche en français. Cohérence interne avec le reste du projet

Tableau 10 : Ensemble des écarts

10.2 Ce qu'on aurait fait différemment

Avec le recul, voici quelques points qu'on aborderait autrement si on devait refaire le projet :

- Utiliser `clock_gettime` dès le début, parce que c'est l'usage moderne et que ça ne coûte presque rien à mettre en place
- Prévoir un fichier de tests automatisés (un script shell qui lance plusieurs scénarios et vérifie les codes retour) plutôt que de faire des tests à la main
- Mettre en place Valgrind tôt dans le projet, pour s'assurer qu'on n'introduit pas de fuites au fur et à mesure
- Séparer plus nettement les statistiques dans leur propre module, plutôt que de les éparpiller entre `calcul.c` et `affichage.c`

10.3 Bilan global

Malgré ces écarts, le projet remplit ses objectifs principaux : myping construit et envoie des paquets ICMP corrects, reçoit et décode les réponses, calcule des statistiques cohérentes avec ping natif, et gère les cas limites de manière propre. Les fonctionnalités absentes (IPv6, flood, export) sont des extensions qu'on n'avait pas listées comme obligatoires.

12. Références bibliographiques

Les sources suivantes ont été consultées pendant la phase de conception et d'implémentation du projet :

- [1] Postel, J. (1981). RFC 792 - Internet Control Message Protocol. IETF. <https://www.rfc-editor.org/rfc/rfc792>
- [2] Muuss, M. (1983). The Story of the PING Program. USENET / BRL Technical Note. <https://ftp.arl.army.mil/~mike/ping.html>
- [3] Forouzan, B. A. (2012). Data Communications and Networking, 5th edition. McGraw-Hill. ISBN 978-0073376226.
- [4] Kerrisk, M. (2010). The Linux Programming Interface. No Starch Press. ISBN 978-1593272203.
- [5] Stevens, W. R. (2003). UNIX Network Programming, Volume 1: The Sockets Networking API, 3rd edition. Addison-Wesley. ISBN 978-0131411555.
- [6] iputils Project (2023). iputils, Network monitoring tools (ping, tracepath, arping). GitHub. <https://github.com/iputils/iputils>
- [7] Linux man-pages (2023). gettimeofday(2), getaddrinfo(3), getopt(3), socket(2), select(2). Disponibles sur <https://man7.org/linux/man-pages/>
- [8] IEEE (2018). The Open Group Base Specifications Issue 7 - POSIX.1-2017. IEEE Std 1003.1. <https://pubs.opengroup.org/onlinepubs/9699919799/>
- [9] Parnas, D. L. (1972). On the Criteria to Be Used in Decomposing Systems into Modules. Communications of the ACM, 15(12), 1053-1058.

Table des matières

GLOSSAIRE	i
LISTE DE TABLEAUX	iii
SOMMAIRE	iv
1. Introduction et présentation du projet.....	5
1.1 Contexte	5
1.2 Objectifs du projet.....	5
1.3 Périmètre	6
2. Analyse de l'existant.....	7
2.1 Un outil ancien, toujours utilisé	7
2.2 Le protocole ICMP en bref	7
2.3 Pourquoi refaire ping ?	8
3. Spécifications fonctionnelles.....	9
3.1 Construction et envoi des paquets ICMP	9
3.2 Réception et traitement des réponses.....	9
3.3 Options de la ligne de commande	10
3.4 Affichage des statistiques.....	11
3.5 Extensions envisagées (non livrées)	11
4. Spécifications non fonctionnelles	12
4.1 Performance	12
4.2 Fiabilité et robustesse.....	12
4.3 Portabilité	13
4.4 Sécurité.....	13
4.5 Maintenabilité	13
5. Architecture et conception	14
5.1 Choix du langage	14
5.2 Décomposition en modules	14
5.3 Variables globales partagées	16
5.4 Flux d'exécution	16
5.5 Diagramme de cas d'utilisation	17
5.6 Diagramme de séquence.....	18
6. Plan de tests et validation	19

6.1 Tests fonctionnels	19
6.2 Tests de robustesse	20
6.3 Comparaison avec ping natif	20
7. Organisation et planification	21
7.1 Composition de l'équipe	21
7.2 Répartition par fichier	22
7.3 Planning prévisionnel	23
7.4 Outils et environnement	24
8. Livrables attendus	25
8.1 Code source	25
8.2 Documentation	25
8.3 Documents	25
9. Identification et gestion des risques	26
10. Écarts entre le cahier des charges initial et la réalisation	27
10.1 Vue d'ensemble des écarts	27
10.2 Ce qu'on aurait fait différemment	29
10.3 Bilan global	29
12. Références bibliographiques	30